

BÀI THỰC HÀNH STRATEGY PATTERN

- ✓ Backend: ASP.NET Core Web API
 - ✓ Frontend: React (gọi API, chọn PayPal hoặc VNPay)
 - ✓ Áp dụng **Strategy Pattern chuẩn GoF**
 - ✓ Runtime chọn thuật toán thanh toán (không dùng Factory)
-

BÀI TOÁN

Hệ thống E-commerce cho phép người dùng chọn phương thức thanh toán. Khác với Factory Pattern: **không tạo object qua factory**, mà **inject tất cả strategy và chọn tại runtime**.

KIẾN TRÚC HỆ THỐNG (STRATEGY)

```
Frontend (React)
    ↓
POST /api/orders/checkout
    ↓
OrderService
    ↓
IPaymentStrategy (Strategy)
    ↓
PaypalStrategy | VNPayStrategy
```

 Điểm khác biệt:

- Không có Factory
 - Không có Creator
 - Chỉ có **Strategy + Context**
-

BACKEND – ASP.NET Core 8 Web API

1 Tạo project

```
dotnet new webapi -n PaymentStrategyDemo
cd PaymentStrategyDemo
```

Cấu trúc thư mục

```
PaymentStrategyDemo
├── Controllers
│   └── OrdersController.cs
├── Services
│   └── OrderService.cs
├── Strategies
│   ├── IPaymentStrategy.cs
│   ├── PaypalStrategy.cs
│   └── VNPayStrategy.cs
├── Models
│   ├── CheckoutRequest.cs
│   └── PaymentResult.cs
└── Program.cs
```

❁ 1. STRATEGY INTERFACE

Strategies/IPaymentStrategy.cs

```
using PaymentStrategyDemo.Models;

namespace PaymentStrategyDemo.Strategies;

public interface IPaymentStrategy
{
    bool CanHandle(string method);
    Task<PaymentResult> Pay(decimal amount);
}
```

❁ 2. MODEL

Models/PaymentResult.cs

```
namespace PaymentStrategyDemo.Models;

public class PaymentResult
{
    public bool Success { get; set; }
    public string Message { get; set; }
}
```

❁ 3. CONCRETE STRATEGIES

Strategies/PaypalStrategy.cs

```

using PaymentStrategyDemo.Models;

namespace PaymentStrategyDemo.Strategies;

public class PaypalStrategy : IPaymentStrategy
{
    public bool CanHandle(string method)
        => method.ToLower() == "paypal";

    public async Task<PaymentResult> Pay(decimal amount)
    {
        await Task.Delay(500);

        return new PaymentResult
        {
            Success = true,
            Message = $"Paid {amount} via PayPal
(Strategy) "
        };
    }
}

```

Strategies/VNPayStrategy.cs

```

using PaymentStrategyDemo.Models;

namespace PaymentStrategyDemo.Strategies;

public class VNPayStrategy : IPaymentStrategy
{
    public bool CanHandle(string method)
        => method.ToLower() == "vnpay";

    public async Task<PaymentResult> Pay(decimal amount)
    {
        await Task.Delay(500);

        return new PaymentResult
        {
            Success = true,
            Message = $"Paid {amount} via VNPay
(Strategy) "
        };
    }
}

```

🌀 4. CONTEXT (ORDER SERVICE)

👉 Đây chính là **Context** trong **Strategy Pattern**

Services/OrderService.cs

```
using PaymentStrategyDemo.Models;
using PaymentStrategyDemo.Strategies;

namespace PaymentStrategyDemo.Services;

public class OrderService
{
    private readonly IEnumerable<IPaymentStrategy>
    _strategies;

    public OrderService(IEnumerable<IPaymentStrategy>
    strategies)
    {
        _strategies = strategies;
    }

    public async Task<PaymentResult> Checkout(string
    method, decimal amount)
    {
        var strategy = _strategies.FirstOrDefault(s =>
    s.CanHandle(method));

        if (strategy == null)
            throw new Exception("Unsupported payment
    method");

        return await strategy.Pay(amount);
    }
}
```

🌀 5. REQUEST MODEL

Models/CheckoutRequest.cs

```
namespace PaymentStrategyDemo.Models;

public class CheckoutRequest
{
    public string PaymentMethod { get; set; }
    public decimal Amount { get; set; }
}
```

```
}
```

6. CONTROLLER

Controllers/OrdersController.cs

```
using Microsoft.AspNetCore.Mvc;
using PaymentStrategyDemo.Models;
using PaymentStrategyDemo.Services;

namespace PaymentStrategyDemo.Controllers;

[ApiController]
[Route("api/orders")]
public class OrdersController : ControllerBase
{
    private readonly OrderService _orderService;

    public OrdersController(OrderService orderService)
    {
        _orderService = orderService;
    }

    [HttpPost("checkout")]
    public async Task<IActionResult>
    Checkout(CheckoutRequest request)
    {
        var result = await _orderService.Checkout(
            request.PaymentMethod,
            request.Amount);

        return Ok(result);
    }
}
```

7. ĐĂNG KÝ DI

Program.cs

```
using PaymentStrategyDemo.Services;
using PaymentStrategyDemo.Strategies;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllers();
```

```

// đăng ký nhiều strategy
builder.Services.AddTransient<IPaymentStrategy,
PaypalStrategy>();
builder.Services.AddTransient<IPaymentStrategy,
VNPayStrategy>();

builder.Services.AddTransient<OrderService>();

builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

// CORS cho React
builder.Services.AddCors(options =>
{
    options.AddPolicy("AllowReact",
        policy =>
        {
            policy.WithOrigins("http://localhost:3000")
                .AllowAnyHeader()
                .AllowAnyMethod();
        });
});

var app = builder.Build();

if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

app.UseHttpsRedirection();
app.UseAuthorization();
app.UseCors("AllowReact");

app.MapControllers();

app.Run();

```

CHẠY BACKEND

dotnet run

Swagger:

`https://localhost:xxxx/swagger`

FRONTEND – REACT (GIỮ NGUYÊN)

 Không cần thay đổi gì so với Factory Pattern

CHẠY FRONTEND

`npm start`

LƯÒNG XỬ LÝ KHI USER CHỌN PAYPAL

1. User chọn PayPal
2. React gửi:

```
{  
  "paymentMethod": "paypal",  
  "amount": 100  
}
```

3. OrderService:

```
_strategies.FirstOrDefault(...)
```

4. Tìm thấy:

PaypalStrategy

5. Gọi:

```
Pay()
```

KHI CHỌN VNPAY

- VNPayStrategy được chọn
 - Gọi Pay()
-

SO SÁNH NHANH: STRATEGY vs FACTORY

Tiêu chí	Factory Pattern	Strategy Pattern
Mục tiêu	Tạo object	Chọn thuật toán
Có Creator	✓ Có	✗ Không
Có Factory	✓ Có	✗ Không
Runtime behavior	✗ Ít	✓ Linh hoạt
DI nhiều implementation	✗ Không cần	✓ Bắt buộc

BÀI TẬP MỞ RỘNG (CHO SINH VIÊN)

- Thêm:
 - MomoStrategy
 - CryptoPaymentStrategy
- Logging:
 - Log strategy được chọn
- Refactor:
 - Không dùng CanHandle → dùng Dictionary
- Nâng cao:
 - Kết hợp Strategy + Factory